Lab1 实验报告

姓名：朱文凯
学号：23307110192

1. 实验目标

本次 Lab1 的目标是实现一个支持基础整数指令的五级流水线 RISC-V CPU，并通过 `make test-lab1` 测试。
本次实现支持的核心指令包括：

- `addi` 、 `xori` 、 `ori` 、 `andi`
- `add` 、 `sub` 、 `and` 、 `or` 、 `xor`
- `addiw` 、 `addw` 、 `subw`

整体采用五级流水线结构：

- IF：取指
- ID：译码与读寄存器
- EX：执行
- MEM：访存
- WB：写回

2. 总体设计思路

本次实现遵循“先搭骨架，再补功能，最后处理冒险与调试接口”的思路。

2.1 Phase 1：搭建流水线骨架

第一阶段先完成 CPU 的基本框架：

- 定义 PC 和取指接口
- 建立 `IF_ID` 、 `ID_EX` 、 `EX_MEM` 、 `MEM_WB` 四组段间寄存器
- 建立整数寄存器堆
- 建立写回路径

- 接好 Difftest 的基础接口

这一阶段的重点不是“能执行多少指令”，而是保证一条指令能够按流水线阶段稳定流动。

2.2 Phase 2：实现译码与执行

第二阶段实现指令译码和 ALU 运算。

在 ID 阶段，根据 `opcode` 、 `funct3` 、 `funct7` 生成控制信号，并提取：

- `rd`
- `rs1`
- `rs2`
- `imm`
- `alu_op`
- `alu_src`
- `reg_write`

在 EX 阶段，根据控制信号驱动 ALU 完成运算。

对于 `addiw` 、 `addw` 、 `subw` ，采用“先进行 32 位运算，再符号扩展到 64 位”的方法，保证符合 RV64 指令语义。

2.3 Phase 3：解决数据冒险

第三阶段处理流水线中的 RAW 数据冒险。

采用两种手段：

- 转发（Forwarding）
- 阻塞与插入气泡（Stall + Bubble）

转发

对于前一条或前两条指令已经算出结果、但尚未写回寄存器堆的情况，直接从流水线后段把数据前递给 EX 阶段 ALU 输入，而不是等待写回。

本次实现中：

- `EX/MEM` 的结果优先级最高
- `MEM/WB` 的结果优先级次之
- `rd == x0` 时不允许转发

阻塞

对于 Load-Use 冒险，仅靠转发无法解决，因此在 ID 阶段检测：

- 若 EX 阶段指令是 Load
- 且它的 `rd` 与当前 ID 阶段的 `rs1` 或 `rs2` 相同

则：

- 保持 PC
- 保持 `IF_ID`
- 在 `ID_EX` 写入一个 bubble

同时为了避免死锁：

- `EX_MEM` 和 `MEM_WB` 在等待期间仍允许正常推进

3. 模块划分

为了让结构更清晰，最终没有把所有逻辑都堆在 `core.sv` 中，而是拆成了几个相对独立的模块：

- `core.sv`
  - 顶层控制与流水线寄存器
  - 统一处理 `stall` 、取指等待、写回与 Difftest 接线
- `core_regfile.sv`
  - 整数寄存器堆
  - 2 读 1 写
  - 保证 `x0` 恒为 0
- `core_decode.sv`
  - 指令字段提取
  - 立即数生成
  - 控制信号生成
- `core_alu.sv`
  - ALU 组合逻辑
- `core_hazard_unit.sv`
  - Load-Use 冒险检测
- `core_forwarding_unit.sv`
  - 前递选择逻辑

这种划分的好处是：

- 顶层负责时序与流水线推进
- 子模块负责纯组合逻辑
- 更容易定位问题，也便于后续扩展

4. 关键问题与修复过程

在调试过程中，真正困难的部分并不只是 ALU 或 forwarding 本身，而是流水线控制和 Difftest 时序。

4.1 取指握手问题

最开始的主要错误是：

- 5000 周期内没有任何指令提交

根因是取指阶段没有满足实验要求：在 `iresp.data_ok` 到来之前， `ireq.valid` 和 `ireq.addr` 必须保持稳定。

一开始的实现里：

- PC 会继续递增
- 取指地址也随之变化
- 导致取指请求和响应错位

修复方法是增加 `fetch_wait`：

- 当请求已发出但还没收到 `data_ok` 时，前端保持不动
- PC 和 `IF_ID` 暂停推进
- 直到当前指令真正取回后再继续

修完这一步后，第一条指令能够被正确提交。

4.2 重复提交问题

后续又出现寄存器比对错误。继续观察后发现，不是算术运算本身出错，而是：

- `MEM_WB` 在等待期间被保持
- 但 `DifftestInstrCommit.valid` 仍保持有效

这会导致同一条指令被重复提交给 Difftest，使参考模型和 DUT 的执行步数错位。

修复方法是只在真正发生一次有效写回提交时拉高 commit：

- 增加 `commit_valid_wb`
- 避免同一条 WB 指令被重复提交

4.3 程序结束但仿真不停止

再之后，测试程序实际上已经运行到了结尾，NEMU 能看到 `GOOD TRAP` ，但仿真并没有正常结束。

根因是没有正确接 `DifftestTrapEvent` 。

修复方法是：

- 在 WB 阶段识别 trap 指令
- 正确上报：
  - `valid`
  - `code`
  - `pc`
  - `cycleCnt`
  - `instrCnt`

这样仿真框架才能知道 DUT 已经结束运行。

5. Difftest 设计思路

Difftest 的核心不是“多接几个端口”，而是保证“提交时刻的架构状态恰好生效”。

本次实现中主要注意了两点：

5.1 Commit 信号

`DifftestInstrCommit.valid` 不能简单地看寄存器写使能，而应当对应“这一拍确实有一条指令提交到 WB 阶段”。

因此使用了 WB 阶段的有效提交信号。

5.2 寄存器状态旁路

同一个时钟沿，WB 写寄存器堆，而 Difftest 读取寄存器状态。
如果直接把寄存器堆内容接给 Difftest，可能会读到旧值。

因此在连接 `DifftestArchIntRegState` 时加入了旁路：

- 如果当前寄存器正好是本拍 WB 要写的寄存器
- 则优先将 `wb_data` 送给 Difftest
- 否则再使用寄存器堆中保存的值

这样可以保证提交指令和架构寄存器状态保持一致。

6. 最终测试结果

最终运行：

```
make test-lab1
```

输出结果中出现：

```
The first instruction of core 0 has commited. Difftest enabled.
Core 0: HIT GOOD TRAP at pc = 0x80010004
```

说明：

- 第一条指令能够正确提交
- 中间没有寄存器比对错误
- 程序能够运行到正确结束位置
- CPU 已通过 Lab1 基本测试

7. 本次实验的收获

通过这次实验，我对以下内容有了更清楚的理解：

- 五级流水线的基本结构

- 段间寄存器在时序逻辑中的作用
- 指令译码与控制信号设计
- RV64 中 W 型指令的处理方式
- RAW 数据冒险的转发与阻塞机制
- Difftest 的真正关注点不是“接口形式”，而是“提交时序是否正确”

这次实验也让我意识到，流水线 CPU 中很多表面上的“算错”，真实原因往往并不在 ALU，而在：

- 握手协议
- 有效位传播
- 流水线停顿控制
- Difftest 提交时序

8. 大模型的使用说明

本次实验中，我使用了 Cursor / Gemini / NotebookLM 作为辅助工具，主要作用包括：

- 帮助梳理阶段目标与实现顺序
- 协助分析错误日志与定位问题来源
- 帮助整理模块划分与文档总结

但所有关键逻辑，例如：

- 流水线寄存器如何推进
- 数据冒险如何处理
- 取指握手如何修复
- Difftest trap 如何接线

都经过了结合代码与测试结果的人工检查和反复验证，而不是直接照搬生成代码。